

# W8D1 – Esercitazione DVWA e Burp Suite

Studente: Riccardo Luzzi

Corso: Cyber Security

Sistema utilizzato: Kali Linux

Applicazioni utilizzate: DVWA, Burp Suite Community Edition, MariaDB, Apache

Ambiente: macchina virtuale Kali Linux con scheda di rete in modalità bridge

## Introduzione

In questa esercitazione ho configurato DVWA (Damn Vulnerable Web Application) su Kali Linux e l'ho utilizzato insieme a Burp Suite per osservare e modificare le richieste HTTP generate dall'applicazione. Prima di iniziare ho impostato la scheda di rete della macchina virtuale in modalità bridge, così da avere la connessione necessaria per scaricare e preparare l'ambiente. Le prove successive sono state svolte interamente in locale, utilizzando l'indirizzo 127.0.0.1.

Dopo aver completato l'installazione e la configurazione di DVWA, ho verificato il funzionamento del database e del web server. Nella seconda parte ho utilizzato Burp Suite in tre prove diverse: al livello Low ho intercettato e modificato una richiesta di login, mentre ai livelli Medium e High ho lavorato sulla sezione SQL Injection per osservare come cambia la gestione del parametro tra le diverse configurazioni. Il Repeater mi ha permesso di inviare nuovamente le richieste dopo aver modificato soltanto i valori interessati.

Negli screenshot della relazione ho mantenuto solo le informazioni utili all'esercitazione. I valori di sessione non sono rilevanti per la spiegazione e vengono considerati dati temporanei del laboratorio.

## 1. Preparazione di DVWA

### 1.1 Configurazione della rete della macchina virtuale

Prima di iniziare l'installazione ho controllato anche la configurazione di rete della macchina virtuale Kali. In VirtualBox ho impostato la scheda di rete in modalità bridge, in modo che Kali potesse utilizzare la connessione della rete reale e accedere a Internet durante la preparazione dell'ambiente. Questo passaggio è necessario soprattutto per scaricare DVWA da GitHub e installare o aggiornare gli eventuali pacchetti richiesti. Una volta completata la configurazione, invece, le prove tra DVWA e Burp Suite sono state eseguite localmente tramite 127.0.0.1.

### 1.2 Download dell'applicazione

Per prima cosa ho lavorato nella directory /var/www/html, che viene utilizzata da Apache per i contenuti web. Da qui ho scaricato il progetto DVWA e ho verificato che la cartella fosse presente correttamente.

- **git clone <https://github.com/digininja/DVWA.git>**

**Figura 1 – Download di DVWA nella cartella /var/www/html**

```
(kali@kali)-[~]
$ systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; disabled; preset: disabled)
   Active: active (running) since Tue 2026-09-01 04:25:41 EDT; 26s ago
     Invocation: b79f236d41e458c5eeaf180099c
       Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 3671 (apache2)
      Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
        Tasks: 6 (limit: 4450)
       Memory: 20.2M (peak: 20.4M)
          CPU: 89ms
    CGroup: /system.slice/apache2.service
            └─3671 /usr/sbin/apache2 -k start -DFOREGROUND
              3687 /usr/sbin/apache2 -k start -DFOREGROUND
              3689 /usr/sbin/apache2 -k start -DFOREGROUND
              3690 /usr/sbin/apache2 -k start -DFOREGROUND
              3691 /usr/sbin/apache2 -k start -DFOREGROUND
              3692 /usr/sbin/apache2 -k start -DFOREGROUND

Sep 01 04:25:41 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Sep 01 04:25:41 kali apachecl[3671]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.0.1:1. Set the 'ServerName' directive globally to suppress this message
Sep 01 04:25:41 kali systemd[1]: Started apache2.service - The Apache HTTP Server.

(kali@kali)-[~]
$ systemctl status mariadb
● mariadb.service - MariaDB 11.8.6 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; disabled; preset: disabled)
   Active: active (running) since Tue 2026-09-01 04:25:58 EDT; 28s ago
     Invocation: d2c6827185634e10832106c0d0321fa0
       Docs: man:mariadb(8)
            https://mariadb.com/kb/en/library/systemd/
    Process: 3891 ExecStartPre=/bin/sh -c [ -e /usr/bin/galera_recovery ] && VAR= || VAR="/usr/bin/galera_recovery"; [ $? -eq 0 ] && echo _WSREP_START_POSITION=$VAR > /run/mysqld/wsrep-start-position || exit 1 (code=exited, stat>
    Process: 3967 ExecStartPost=/bin/rm -f /run/mysqld/wsrep-start-position /run/mysqld/wsrep-rw-cluster (code=exited, status=0/SUCCESS)
    Process: 3969 ExecStartPost=/etc/mysql/debian-start (code=exited, status=0/SUCCESS)
    Main PID: 3943 (mariadbd)
      Status: "Taking your first requests now..."
        Tasks: 15 (limit: 29411)
       Memory: 168.5M (peak: 173.5M)
          CPU: 970ms
    CGroup: /system.slice/mariadb.service
            └─3943 /usr/sbin/mariadbd

Sep 01 04:25:58 kali mariadbd[3943]: 2026-09-01 4:25:58 0 [Note] InnoDB: Loading buffer pool(s) from /var/lib/mysql/ib_buffer_pool
Sep 01 04:25:58 kali mariadbd[3943]: 2026-09-01 4:25:58 0 [Note] InnoDB: Buffer pool(s) load completed at 260901 4:25:58
Sep 01 04:25:58 kali mariadbd[3943]: 2026-09-01 4:25:58 0 [Note] InnoDB: Server socket created on IP: '127.0.0.1', port: '3306'.
Sep 01 04:25:58 kali mariadbd[3943]: 2026-09-01 4:25:58 0 [Note] mariadbd: Event Scheduler: Loaded 0 events
Sep 01 04:25:58 kali mariadbd[3943]: 2026-09-01 4:25:58 0 [Note] /usr/sbin/mariadbd: ready for connections.
Sep 01 04:25:58 kali mariadbd[3943]: Version: '11.0.6-MariaDB-2 from Debian' socket: /run/mysqld/mysqld.sock port: 3306 -- Please help get to 10k stars at https://github.com/MariaDB/Server
Sep 01 04:25:58 kali systemd[1]: Started mariadb.service - MariaDB 11.8.6 database server.
Sep 01 04:25:58 kali /etc/mysql/debian-start[3971]: Upgrading MariaDB tables if necessary.
Sep 01 04:25:59 kali /etc/mysql/debian-start[3983]: Checking for insecure root accounts.
Sep 01 04:25:59 kali /etc/mysql/debian-start[3987]: Triggering myisam-recover for all MyISAM tables and aria-recover for all Aria tables
lines 1-27/27 (END)
```

## 1.3 Permessi e configurazione dei servizi

Durante la configurazione ho incontrato un problema con i permessi: un primo tentativo di modifica ricorsiva restituiva diversi messaggi “Operation not permitted”. Dopo aver eseguito il comando con i privilegi necessari ho controllato nuovamente la cartella e ho proseguito con la configurazione.

**Figura 2 – Verifica dei permessi della cartella DVWA**

```
(kali@kali)-[/var/www/html]
$ sudo chmod -R 777 DVWA/

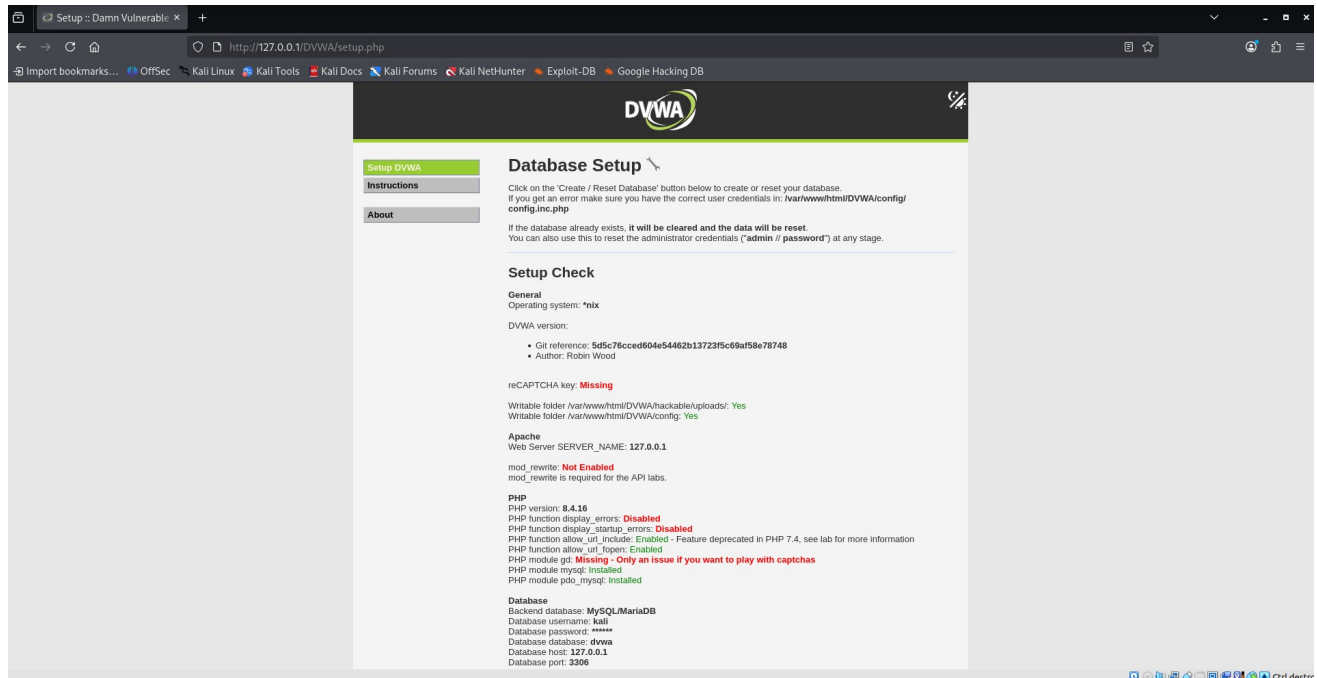
(kali@kali)-[/var/www/html]
$ ls -ld DVWA
drwxrwxrwx 12 root root 4096 Sep  1 04:29 DVWA
```

Dopo aver sistemato i permessi, ho avviato i servizi Apache e MariaDB necessari al funzionamento di DVWA. Successivamente ho configurato il database creando l'utente utilizzato dall'applicazione e ho verificato che i parametri presenti nel file di configurazione di DVWA corrispondessero a quelli impostati nel database.

## 1.4 Creazione del database DVWA

Una volta avviati i servizi ho aperto DVWA dal browser e sono entrato nella pagina di setup. La pagina permette di controllare i requisiti e di creare o reimpostare il database. Dopo la creazione, l'applicazione mi ha riportato alla schermata di autenticazione.

**Figura 3 – Pagina di Setup / Reset Database di DVWA**



## 2. Accesso e verifica dell'applicazione

Dopo la creazione del database ho effettuato l'accesso con le credenziali predefinite del laboratorio. Una volta entrato ho verificato che il menu laterale mostrasse correttamente le diverse vulnerabilità disponibili e la pagina DVWA Security per cambiare il livello di sicurezza.

**Figura 4 – Schermata di accesso a DVWA**

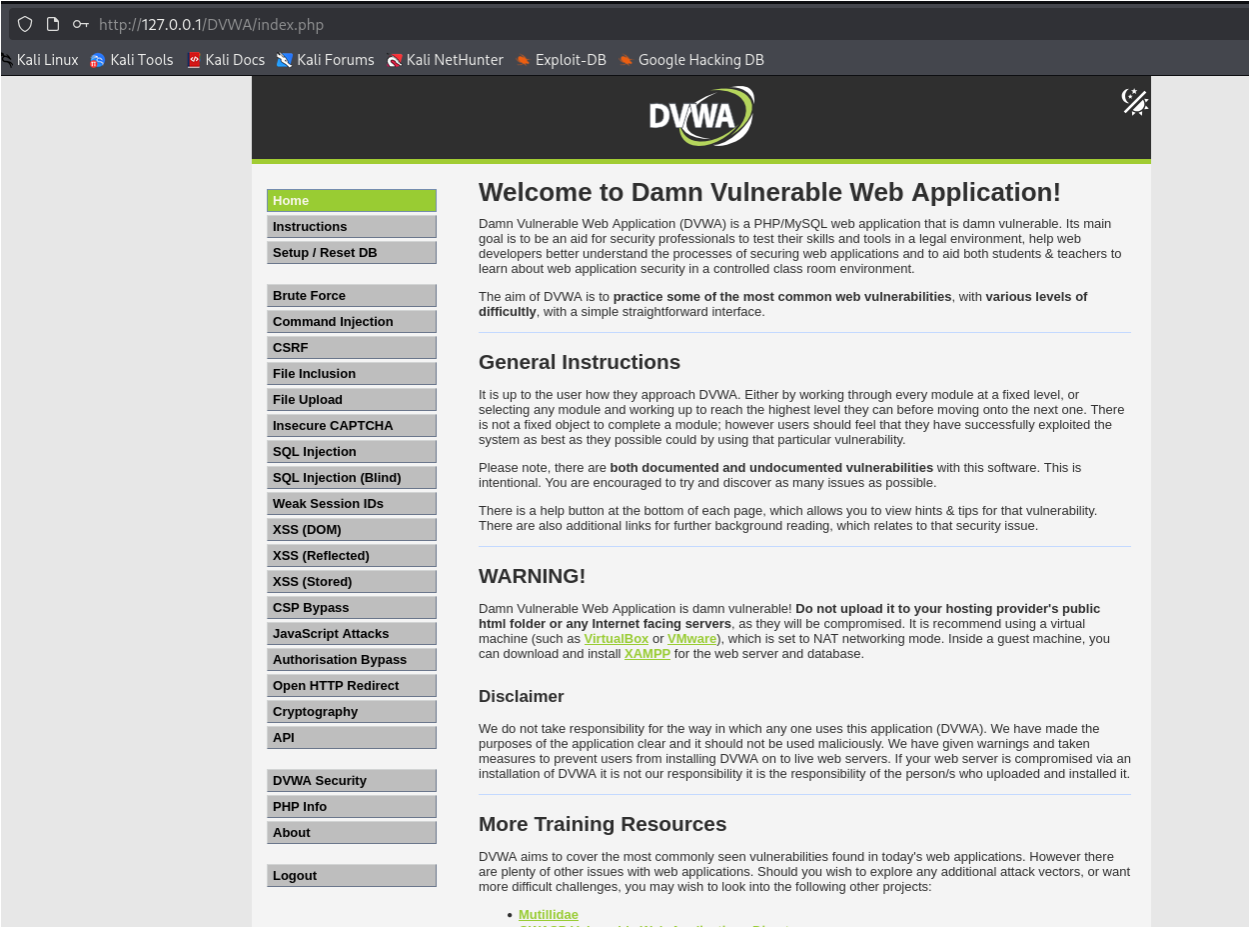


Username

Password

Login

Figura 5 – Pagina principale di DVWA dopo l'accesso



### 3. Configurazione di Burp Suite

Per analizzare le richieste generate da DVWA ho avviato Burp Suite Community Edition e ho utilizzato il browser configurato per passare attraverso il proxy locale di Burp. In questo modo ho potuto visualizzare le richieste HTTP inviate dall'applicazione.

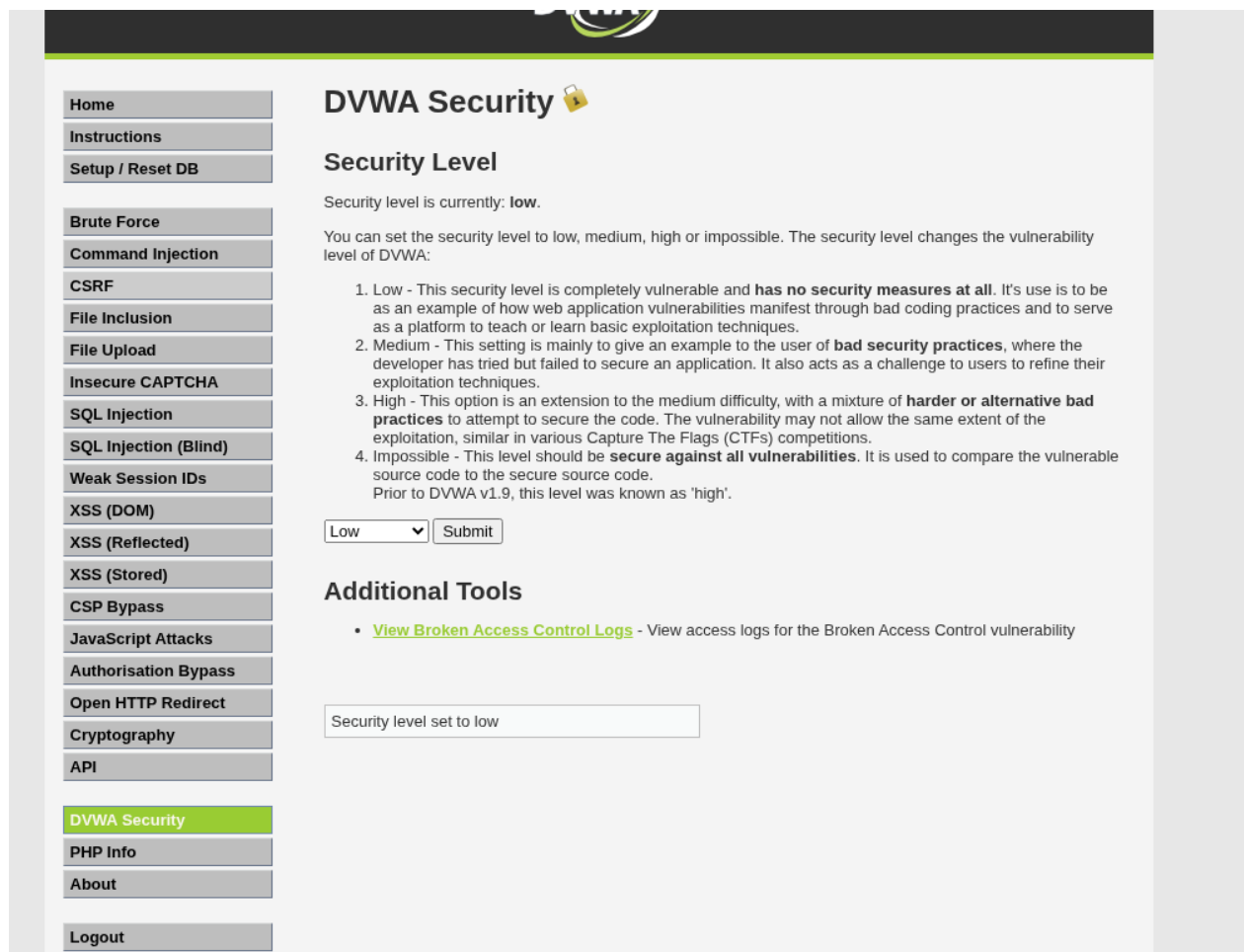
Inizialmente ho utilizzato la funzione Intercept per verificare che Burp riuscisse a catturare correttamente il traffico. Successivamente ho utilizzato soprattutto la sezione HTTP history, dove rimangono visibili le richieste già effettuate. Da qui ho potuto selezionare quelle che mi interessavano e inviarle al Repeater per modificarle e ripeterle durante le prove.

### 4. Burp Suite – livello Low: modifica della richiesta di login

#### 4.1 Impostazione del livello di sicurezza

Per la prima prova ho impostato DVWA sul livello di sicurezza Low dalla pagina DVWA Security. In questo modo ho potuto lavorare con l'applicazione nel livello previsto dall'esercitazione e verificare, anche nelle richieste intercettate da Burp Suite, il valore security=low.

Figura 6 – Impostazione del livello di sicurezza Low



## 4.2 Intercettazione della richiesta di login

Successivamente sono tornato alla pagina di login di DVWA e ho effettuato l'accesso utilizzando le credenziali predefinite del laboratorio. Con Burp Suite ho intercettato la richiesta POST diretta a /DVWA/login.php. Nel corpo della richiesta sono visibili i parametri username=admin e password=password, mentre nel cookie compare security=low.

Questa richiesta mi è servita come base per la prova successiva, perché mostra esattamente i dati inviati dal browser al server durante il tentativo di autenticazione.

**Figura 7 – Richiesta POST di login intercettata con le credenziali originali**



```
1 POST /DVWA/login.php HTTP/1.1
2 Host: 127.0.0.1
3 Content-Length: 88
4 Cache-Control: max-age=0
5 sec-ch-ua: "Chromium";v="145", "Not:A-Brand";v="99"
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: "Linux"
8 Accept-Language: en-US,en;q=0.9
9 Origin: http://127.0.0.1
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: http://127.0.0.1/DVWA/login.php
19 Accept-Encoding: gzip, deflate, br
20 Cookie: PHPSESSID=17b3b975b71757cd2ad3766f206d4c5d; security=low
21 Connection: keep-alive
22
23 username=admin&password=password&Login=Login&user_token=d1a436a3bef3b20ca01156e45d0ec52e
```

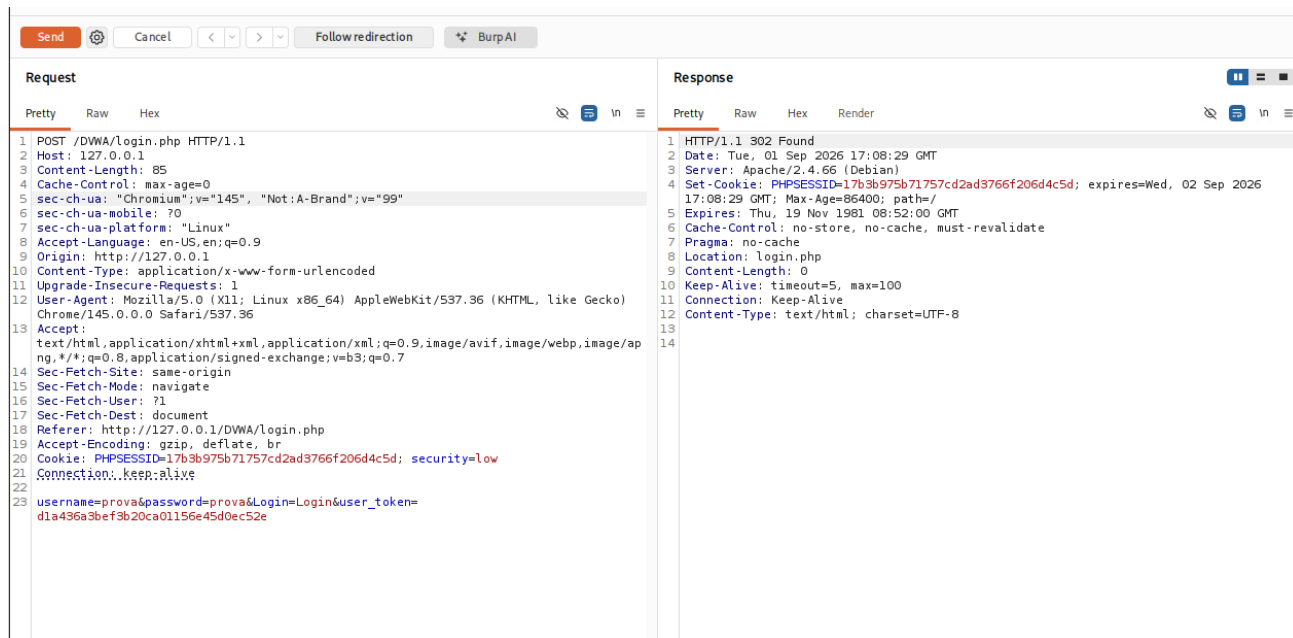
## 4.3 Modifica delle credenziali nel Repeater

Ho quindi inviato la richiesta al Repeater e ho modificato solamente i valori dei campi username e password, sostituendo admin/password con prova/prova. In questo modo ho potuto ripetere la stessa richiesta senza dover tornare ogni volta al form di login e osservare direttamente la risposta del server.

- **username=prova&password=prova**

Dopo aver premuto Send, il server ha restituito una risposta HTTP 302 Found con reindirizzamento verso login.php. Il codice 302, da solo, indica il redirect; per capire l'esito effettivo del tentativo è stato quindi necessario controllare anche la pagina restituita dopo il reindirizzamento.

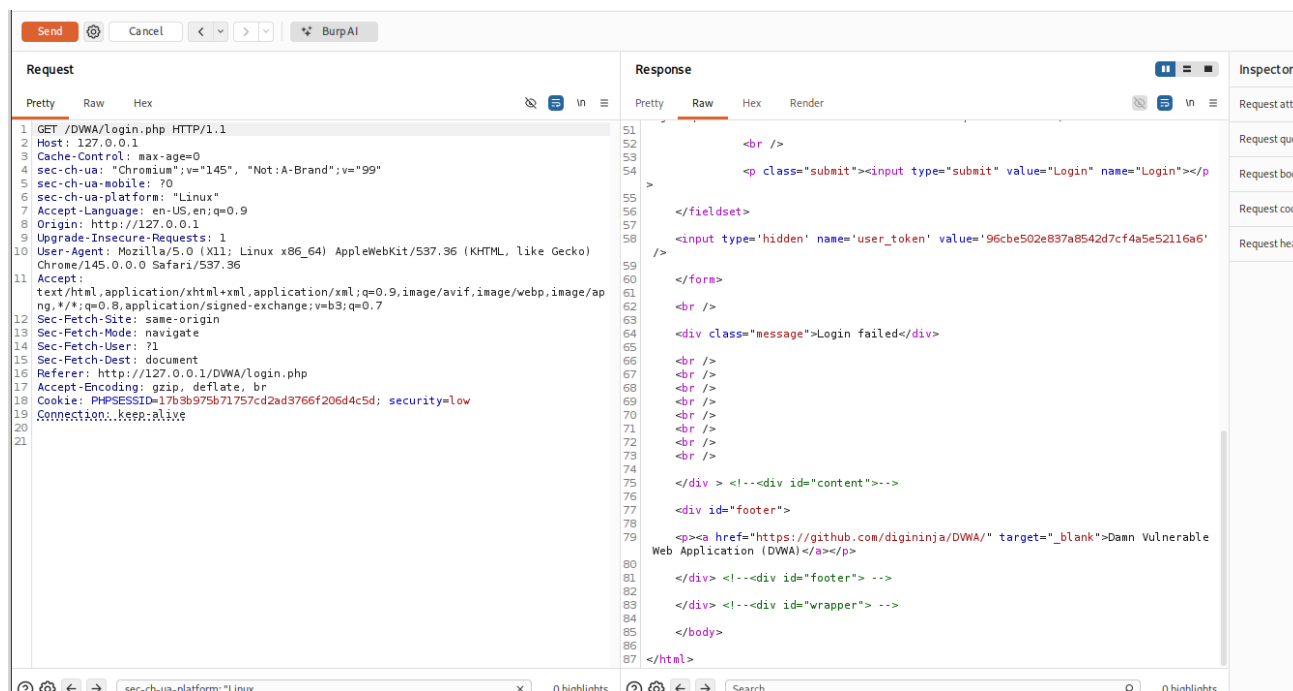
**Figura 8 – Credenziali modificate nel Burp Repeater e risposta HTTP 302**



## 4.4 Verifica del risultato

Seguendo il reindirizzamento, nella risposta della pagina di login compare il messaggio “Login failed”. Questo conferma che i valori prova/prova inviati tramite il Repeater non vengono accettati come credenziali valide. La prova mi ha permesso di vedere in modo pratico come Burp Suite consenta di intercettare una richiesta reale, modificarne i parametri e verificarne il risultato dalla risposta HTTP.

**Figura 9 – Messaggio Login failed dopo la modifica delle credenziali**



## 5. SQL Injection – livello Medium

Dopo aver completato il livello Low ho impostato DVWA su Medium. In questo livello ho utilizzato Burp Suite per vedere cosa viene realmente inviato al server quando seleziono un ID e premo Submit. La richiesta intercettata è di tipo POST e il valore non compare nell'indirizzo della pagina, ma nel corpo della richiesta. Qui si può vedere il parametro id insieme a Submit; nel cookie compare anche security=medium, quindi ho verificato di stare lavorando sul livello corretto.

Dopo aver individuato la richiesta l'ho inviata al Repeater. Sono partito dal valore normale id=1 e ho modificato soltanto quel parametro usando la condizione 1 OR 1=1. Ho scelto questa prova perché 1=1 è sempre vera: aggiungendola con OR posso controllare se l'applicazione usa direttamente il valore ricevuto nella query senza gestirlo in modo sicuro. Ho utilizzato questa forma senza apici perché, in questo livello, il parametro id viene gestito come un valore numerico.

### • id=1 OR 1=1&Submit=Submit

Dopo aver premuto Send, Burp ha inviato nuovamente la stessa richiesta con il parametro modificato. Il server ha risposto con HTTP 200 OK, ma la parte più importante è il contenuto della risposta: DVWA non restituisce più soltanto l'utente associato all'ID 1, ma mostra diversi record presenti nel database, tra cui admin, Gordon Brown, Hack Me, Pablo Picasso e Bob Smith. Questo risultato conferma che la condizione 1 OR 1=1, essendo sempre vera, ha modificato il comportamento della query. Il Repeater mi ha quindi permesso di cambiare direttamente il parametro e verificare il risultato senza dover ripetere ogni volta l'inserimento dal browser.

**Figura 10 – Richiesta POST intercettata con Burp Suite al livello Medium**

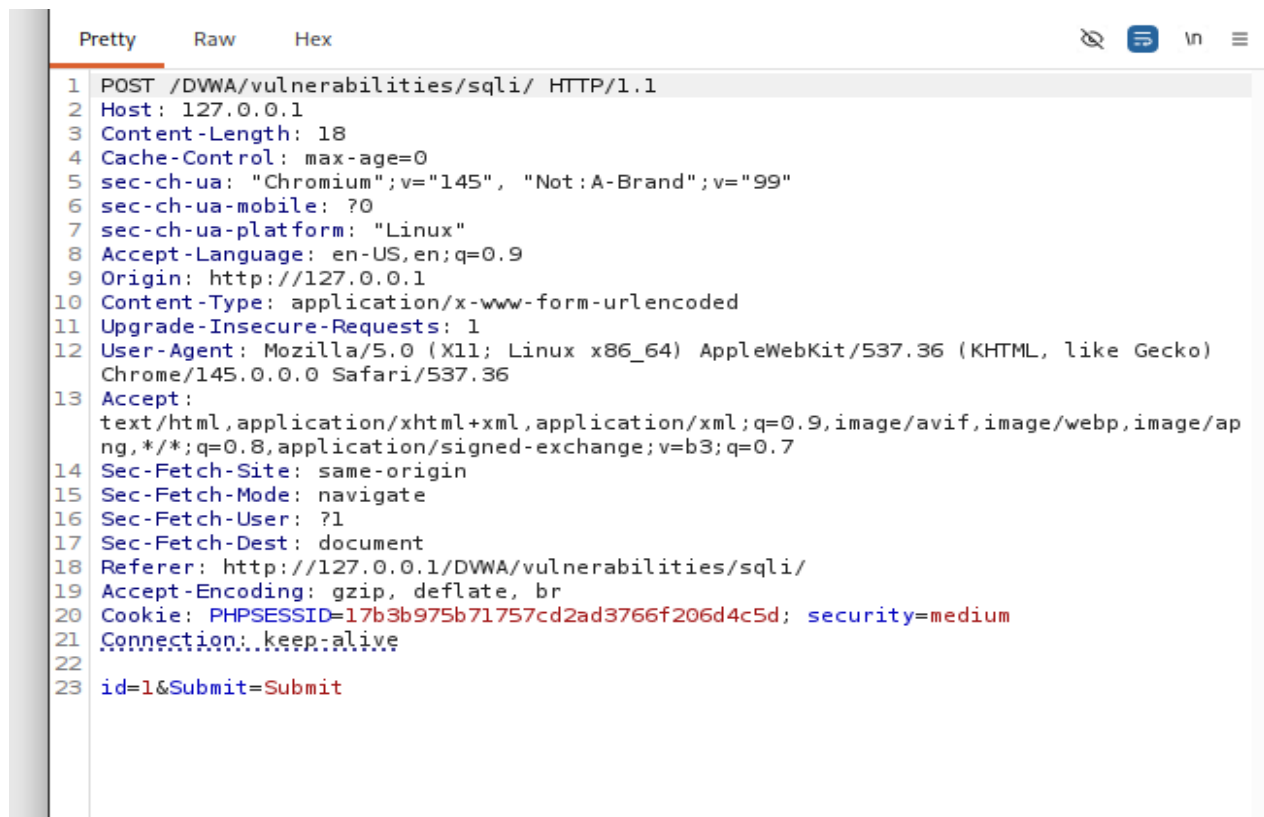


Figura 11 – SQL Injection con 1 OR 1=1 e visualizzazione di più record nel Burp Repeater

Send Cancel < > BurpAI

**Request**

Pretty Raw Hex

```
1 POST /DWA/vulnerabilities/sqli/ HTTP/1.1
2 Host: 127.0.0.1
3 Content-Length: 25
4 Cache-Control: max-age=0
5 sec-ch-ua: "Chromium";v="145", "Not:A-Brand";v="99"
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: "Linux"
8 Accept-Language: en-US,en;q=0.9
9 Origin: http://127.0.0.1
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0
13 Safari/537.36
14 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
15 Sec-Fetch-Site: same-origin
16 Sec-Fetch-Mode: navigate
17 Sec-Fetch-Dest: document
18 Referer: http://127.0.0.1/DWA/vulnerabilities/sqli/
19 Accept-Encoding: gzip, deflate, br
20 Cookie: PHPSESSID=17b3b975b71757cd2ad3766f206d4c5d; security=medium
21 Cookie: PHPSESSID=17b3b975b71757cd2ad3766f206d4c5d; security=medium
22
23 id=1<OR>1=1&Submit=Submit
```

**Response**

Pretty Raw Hex Render

```
85 <option value="2">
86 2
87 </option>
88 <option value="3">
89 3
90 </option>
91 <option value="4">
92 4
93 </option>
94 <option value="5">
95 5
96 </option>
97 </select>
98 <input type="submit" name="Submit" value="Submit">
99 </p>
100 </form>
101 <pre>
102 ID: 1 OR 1=1<br />
103 First name: admin<br />
104 Surname: admin
105 </pre>
106 <pre>
107 ID: 1 OR 1=1<br />
108 First name: Gordon<br />
109 Surname: Brown
110 </pre>
111 <pre>
112 ID: 1 OR 1=1<br />
113 First name: Hack<br />
114 Surname: Me
115 </pre>
116 <pre>
117 ID: 1 OR 1=1<br />
118 First name: Pablo<br />
119 Surname: Picasso
120 </pre>
121 <pre>
122 ID: 1 OR 1=1<br />
123 First name: Bob<br />
124 Surname: Smith
125 </pre>
126 </div>
127 </h2>
128 More Information
```

**Inspector**

Request attribut

Request query p

Request body pa

Request cookies

Request header

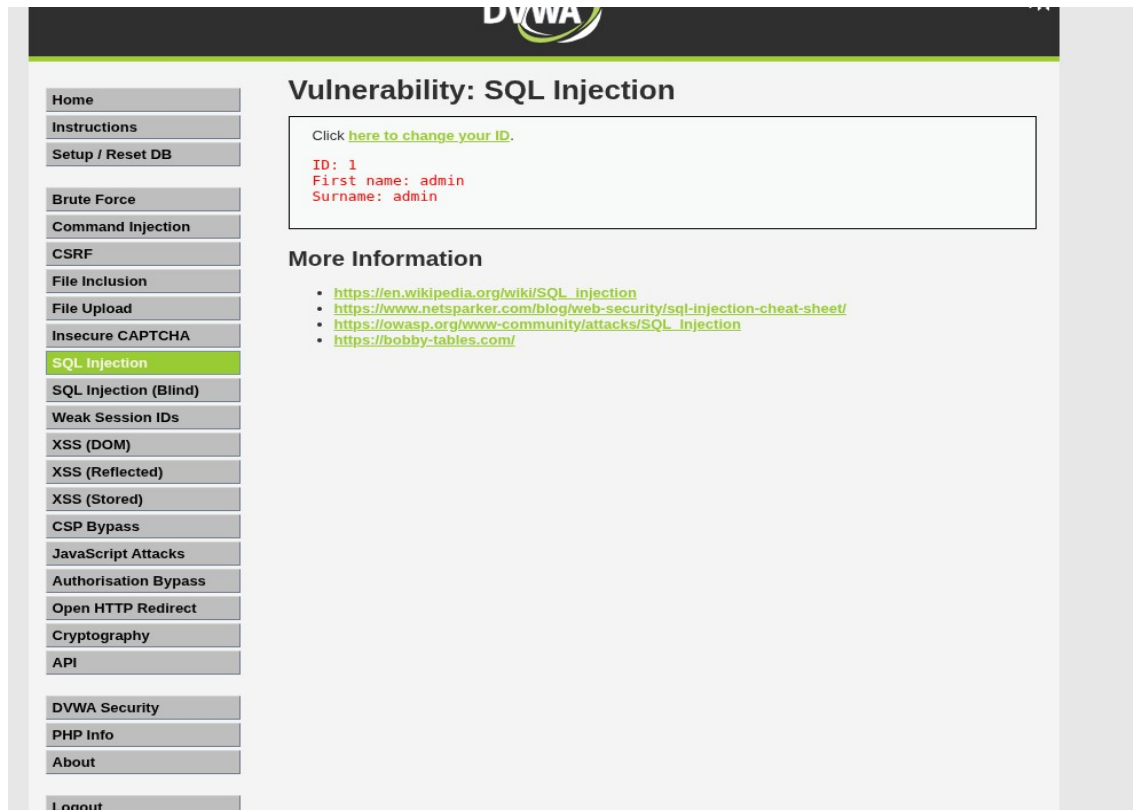
Response head

## 6. SQL Injection – livello High

### 6.1 Verifica con un ID valido

Impostando il livello High ho notato subito una differenza rispetto ai livelli precedenti. Il campo User ID non compare direttamente nella pagina principale, ma viene mostrato il collegamento “here to change your ID”, che apre una finestra separata. Come prima prova ho inserito semplicemente il valore 1, così da controllare il comportamento normale prima di analizzare la richiesta con Burp Suite. Dopo l’invio, DVWA ha restituito un solo record, relativo all’utente admin.

**Figura 12 – Risultato ottenuto con User ID 1 al livello High**

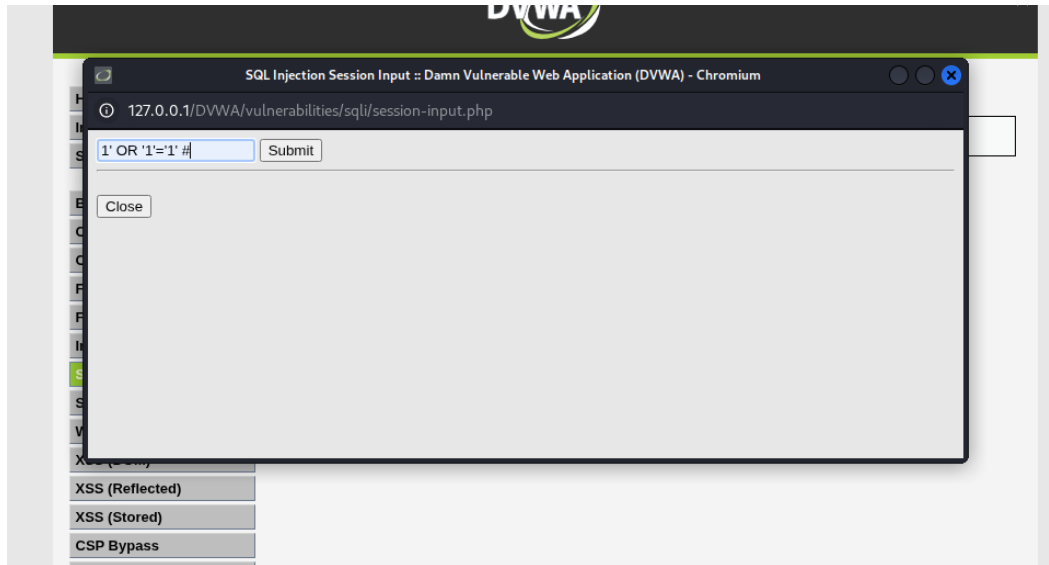


### 6.2 Intercettazione della richiesta con Burp Suite

Dopo aver verificato il comportamento normale con l’ID 1, ho aperto nuovamente la finestra “SQL Injection Session Input”. In questa prova ho inserito il payload 1' OR '1'='1' #. La condizione '1'='1' è sempre vera, mentre il carattere # serve a commentare la parte successiva della query. In questo modo ho potuto verificare come viene gestito il parametro ID al livello High.

Dopo l’invio ho controllato la richiesta generata con Burp Suite. La richiesta è di tipo POST ed è diretta a /DVWA/vulnerabilities/sql/session-input.php; nel cookie compare security=high, quindi ho verificato di stare lavorando sul livello corretto. Il valore inserito viene trasmesso nel corpo della richiesta e Burp lo mostra in forma URL-encoded.

**Figura 13 – Inserimento del payload SQL Injection nella finestra Session Input al livello High**

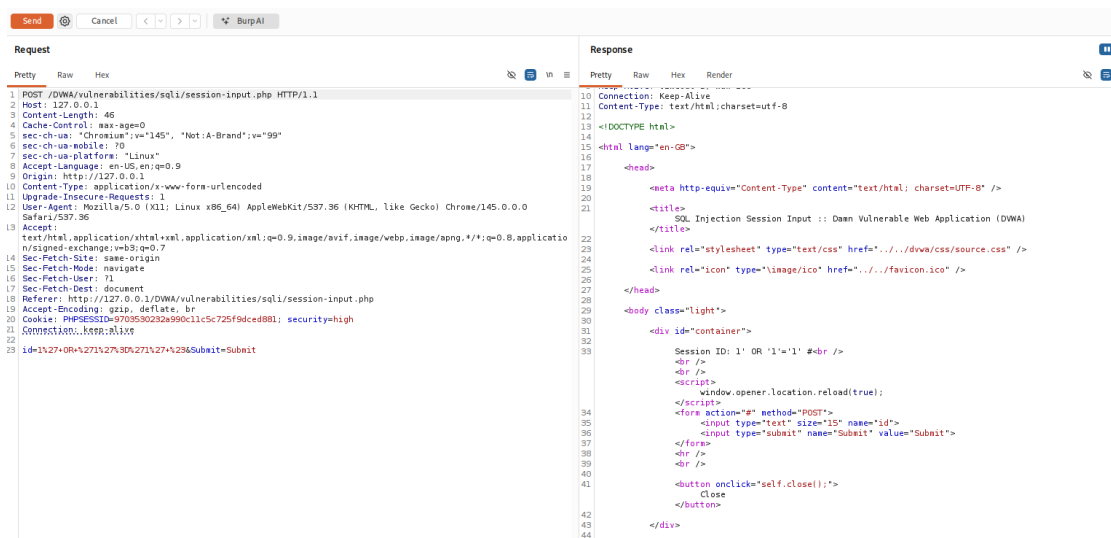


### 6.3 Verifica con Burp Repeater

Nel Repeater ho utilizzato la richiesta generata dalla finestra Session Input e ho controllato il parametro id. Il payload compare in forma URL-encoded come `id=1%27+OR+%271%27%3D%271%27+%23&Submit=Submit`; una volta decodificato corrisponde alla stringa `1' OR '1'='1' #` inserita nella finestra. Ho quindi inviato la richiesta con Send per verificare come DVWA avrebbe gestito il valore modificato.

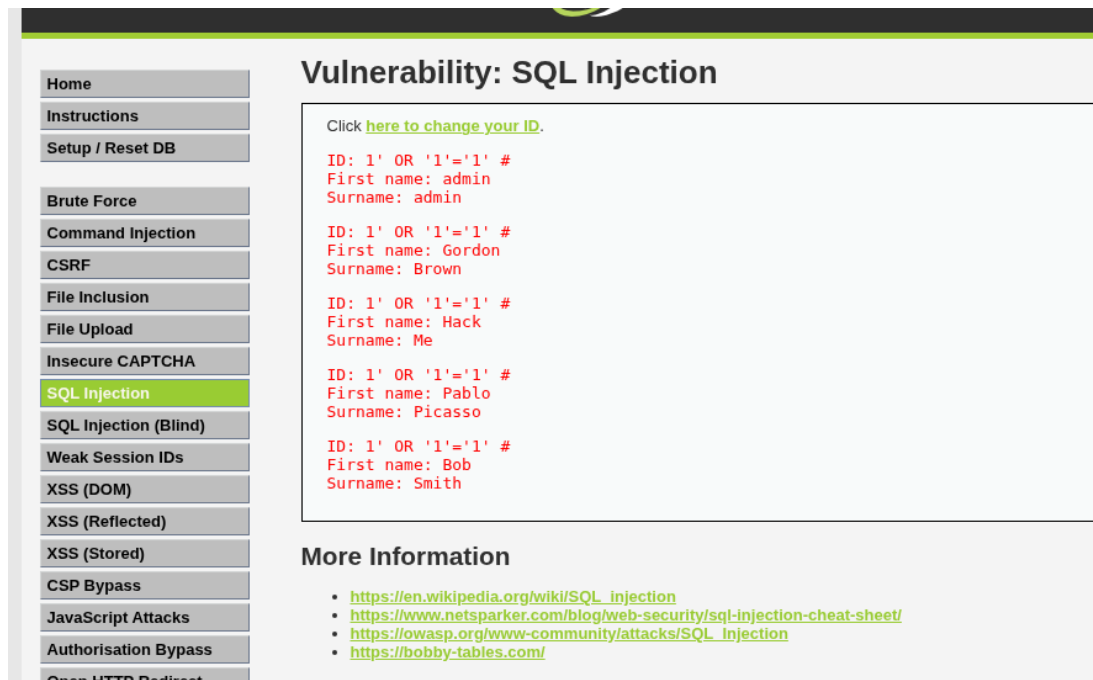
Dopo aver premuto Send, il server ha risposto con HTTP 200 OK. Nella Response di Burp è visibile la voce "Session ID: 1' OR '1'='1' #", che conferma che il valore modificato è stato ricevuto da session-input.php e salvato nella sessione. A differenza del livello Medium, quindi, nel livello High il parametro viene prima gestito dalla finestra separata e memorizzato nella sessione, per poi essere utilizzato dalla pagina principale della SQL Injection.

**Figura 14 – Payload inviato nel Burp Repeater e valore memorizzato nella Session ID**



Dopo aver inviato il payload tramite Burp Repeater, sono tornato alla pagina principale della vulnerabilità SQL Injection. La pagina ha utilizzato il valore salvato nella sessione e ha restituito più record del database: admin, Gordon Brown, Hack Me, Pablo Picasso e Bob Smith. Il risultato conferma che la condizione inserita ha modificato la query e ha permesso di ottenere più risultati rispetto al normale utilizzo dell'ID.

**Figura 15 – Risultato della SQL Injection al livello High con visualizzazione di più record**



## 7. Confronto tra le prove

Le tre prove hanno mostrato modalità di utilizzo diverse. Al livello Low ho lavorato sulla richiesta di login, modificando username e password nel Repeater e verificando il messaggio di autenticazione fallita. Al livello Medium ho analizzato la richiesta POST della SQL Injection e ho modificato il parametro id con 1 OR 1=1, ottenendo più record nella risposta. Al livello High ho invece utilizzato la finestra Session Input: il payload è stato inviato a session-input.php, salvato nella sessione e poi utilizzato dalla pagina principale, che ha restituito anche in questo caso più record del database.

La parte più utile dell'esercitazione è stata proprio confrontare il comportamento delle richieste. Guardando solo l'interfaccia di DVWA alcune differenze sono meno evidenti, mentre con Burp Suite si vede quali endpoint vengono chiamati, quale metodo HTTP viene utilizzato e dove viene inserito il parametro che interessa.

## 8. Conclusioni

Nel complesso sono riuscito a configurare DVWA su Kali Linux, collegarlo al database e utilizzare Burp Suite per analizzare il traffico locale. Nella prova Low ho osservato e modificato una richiesta di autenticazione, mentre nelle prove Medium e High ho lavorato sulla SQL Injection verificando non solo la richiesta inviata, ma anche il risultato ottenuto dopo la modifica del parametro id.

Durante l'attività ho incontrato anche alcuni problemi pratici, soprattutto con i permessi della cartella DVWA. Risolverli mi è servito per capire meglio cosa stavo facendo invece di limitarmi a ripetere i passaggi. Burp Repeater è stato particolarmente utile perché consente di partire da una richiesta reale, modificarne un solo parametro e inviarla di nuovo senza dover ripetere ogni volta tutta la navigazione dal browser.

L'esercitazione è stata svolta esclusivamente sulla macchina locale e su DVWA, applicazione pensata per l'apprendimento delle vulnerabilità web. La prova mi ha permesso di vedere concretamente come Burp Suite intercetta e modifica le richieste HTTP e come cambia il flusso tra i diversi livelli: nel Medium il parametro viene inviato direttamente nella richiesta POST, mentre nell'High passa prima da session-input.php e viene salvato nella sessione prima di essere utilizzato dalla pagina principale.